

ĐẠI HỌC BÁCH KHOA HÀ NỘI

*

BÁO CÁO TIẾN ĐỘ HỌC PHẦN
PROJECT 1



SOICT

TÊN ĐỀ TÀI
HỆ THỐNG QUẢN LÝ CÔNG VIỆC

Giảng viên hướng dẫn : Th.S Phạm Quang Hiếu
Sinh viên thực hiện : Nguyễn Trọng Nhân – 202490073
Lớp : CNTT 1.1 - K69

TUẦN 05
Hà Nội 05 - 2026

Mục lục

1	CHƯƠNG 1: KẾT QUẢ THỰC HIỆN TRONG TUẦN 5	2
	(27/04/2026 - 03/05/2026)	
1.1	Tổng quan	2
1.2	Quản lý Card	2
1.3	Quản lý Checklist	7
1.4	Kết luận	11
2	CHƯƠNG 2: KẾ HOẠCH PHÁT TRIỂN TUẦN 6	11
2.1	Mục tiêu	11
2.2	Triển khai chi tiết	11

1 CHƯƠNG 1: KẾT QUẢ THỰC HIỆN TRONG TUẦN 5 (27/04/2026 - 03/05/2026)

1.1 Tổng quan

Trong tuần thứ 5 của dự án (27/04/2026 - 03/05/2026), các công việc chính tập trung vào việc phát triển đầy đủ các chức năng quản lý Card và Checklist thuộc Giai đoạn 2 (Core Features), đồng thời tiến hành chuẩn hóa và tối ưu hóa toàn bộ hệ thống API.

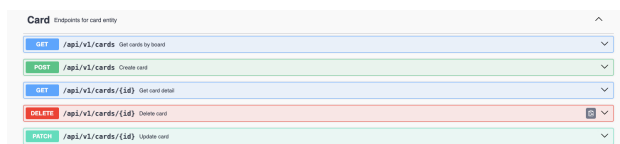
- Phát triển đầy đủ các API CRUD cho Card, bao gồm các thuộc tính mở rộng như description, assignee và due date.
- Xây dựng logic xử lý position của Card trong List nhằm hỗ trợ chức năng drag-and-drop.
- Triển khai hoàn chỉnh các API Checklist và Checklist Item trong Card.

Tóm tắt kết quả tuần 5

Hạng mục	Kết quả đạt được
Quản lý Card	Xây dựng hoàn chỉnh các API CRUD cho Card, bao gồm các thuộc tính mở rộng (description, assignee, due date) và logic xử lý position phục vụ drag-and-drop
Quản lý Checklist	Triển khai đầy đủ các API cho Checklist và Checklist Item trong Card, bao gồm tạo, cập nhật trạng thái và xóa

1.2 Quản lý Card

- Tổng hợp API Card (Create / Read / Update / Delete)



The screenshot shows a list of API endpoints for the 'Card' entity. The endpoints are: GET /api/v1/cards (Get cards by board), POST /api/v1/cards (Create card), GET /api/v1/cards/{id} (Get card detail), DELETE /api/v1/cards/{id} (Delete card), and PATCH /api/v1/cards/{id} (Update card). Each endpoint is represented by a colored bar with a corresponding HTTP method icon.

Card Endpoints for card entity	
GET	/api/v1/cards Get cards by board
POST	/api/v1/cards Create card
GET	/api/v1/cards/{id} Get card detail
DELETE	/api/v1/cards/{id} Delete card
PATCH	/api/v1/cards/{id} Update card

Hình 1: Card API

- Thiết kế entity/model và quan hệ cho Card

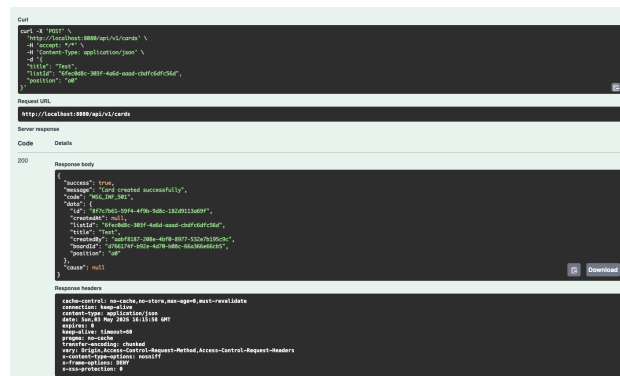
- Thiết kế quan hệ: Card thuộc về List (@ManyToOne); thành viên được assign qua bảng trung gian CardMember.
- Triển khai chức năng tạo Card (TC17)
 - Xây dựng API tạo Card trong List (POST /api/v1/cards).
 - * Nhận thông tin từ client: `listId`, `title`, `position`.
 - * Xác thực người dùng thông qua access token.
 - * Kiểm tra List tồn tại và user có quyền truy cập.
 - * Trả về thông tin card vừa tạo (`cardId`, `title`, `listId`, `position`).
 - Áp dụng validation: `title`, `listId`, `position` không được để trống; trả về HTTP 400 nếu thiếu.
- Triển khai chức năng xem chi tiết Card (TC18)
 - Xây dựng API lấy chi tiết Card (GET /api/v1/cards/:id).
 - * Nhận `cardId` từ path parameter.
 - * Kiểm tra card tồn tại, trả về HTTP 404 nếu không tìm thấy.
 - * Trả về đầy đủ thông tin card bao gồm `title`, `description`, `startDate`, `dueDate`, `position`, `isCompleted`.
- Triển khai chức năng cập nhật Card (TC19 - TC20)
 - Xây dựng API cập nhật thông tin Card (PATCH /api/v1/cards/:id).
 - * Cập nhật các trường: `title`, `description`, `startDate`, `dueDate`, `position`, `listId`, `isCompleted`.
 - * Hỗ trợ di chuyển card giữa các List thông qua việc cập nhật `listId` và `position` đồng thời (phục vụ drag-and-drop).
 - * Kiểm tra card tồn tại trước khi cập nhật, trả về HTTP 404 nếu không tìm thấy.
 - * Trả về HTTP 200 và thông tin card sau khi cập nhật.
- Triển khai chức năng xóa Card (TC21)
 - Xây dựng API xóa Card (DELETE /api/v1/cards/:id).
 - * Kiểm tra card tồn tại, trả về HTTP 404 nếu không tìm thấy.
 - * Trả về HTTP 200 khi xóa thành công.
- Triển khai API lấy danh sách Card theo Board (TC22)
 - Xây dựng API (GET /api/v1/cards?boardId=).

- * Nhận boardId từ query parameter.
- * Trả về toàn bộ card thuộc Board, sắp xếp theo position.
- * Hỗ trợ trường hợp Board chưa có card (trả về danh sách rỗng).

- **Kết quả kiểm thử chức năng Card**

- **TC17 - Tạo Card thành công**

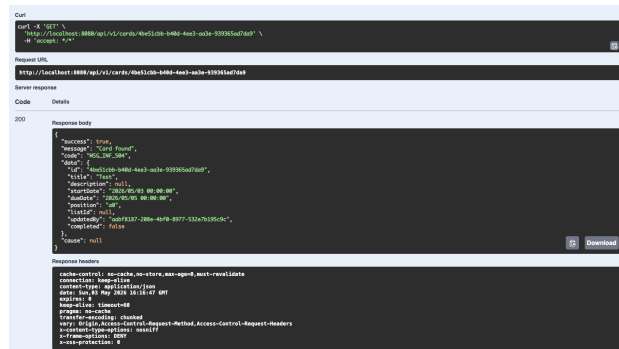
- * API: POST /api/v1/cards
- * Input: listId, title = “Task 1”, position
- * Pre-condition: User đã xác thực, List tồn tại
- * Expected result: HTTP 200 OK, response chứa cardId, title, listId, position



Hình 2: Triển khai API Card TC17

- **TC18 - Lấy chi tiết Card thành công**

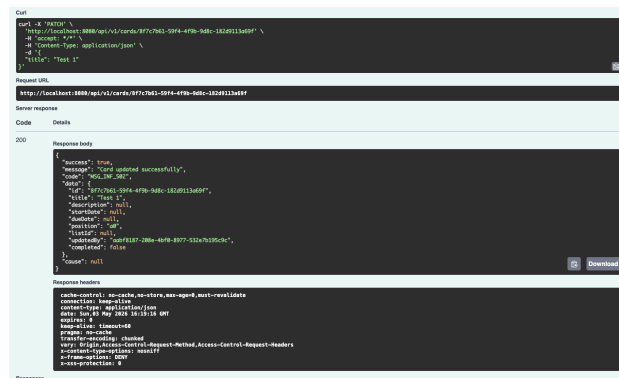
- * API: GET /api/v1/cards/:id
- * Input: cardId hợp lệ
- * Pre-condition: Card tồn tại, user đã xác thực
- * Expected result: HTTP 200 OK, response chứa đầy đủ thông tin card



Hình 3: Triển khai API Card TC18

– TC19 - Cập nhật thông tin Card thành công

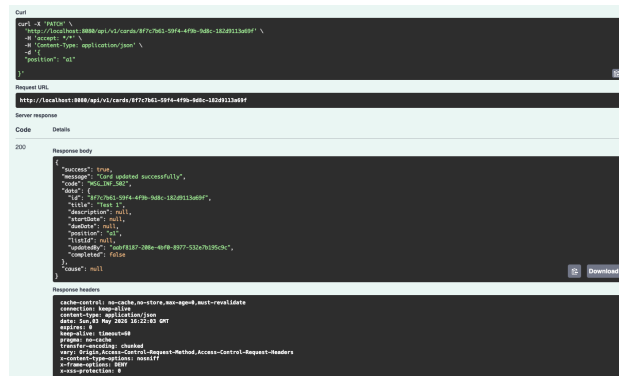
- * API: PATCH /api/v1/cards/:id
- * Input: title, description, startDate, dueDate, isCompleted
- * Pre-condition: Card tồn tại, user đã xác thực
- * Expected result: HTTP 200 OK, thông tin card được cập nhật đầy đủ



Hình 4: Triển khai API Card TC19

– TC20 - Cập nhật vị trí Card giữa các List (drag-and-drop)

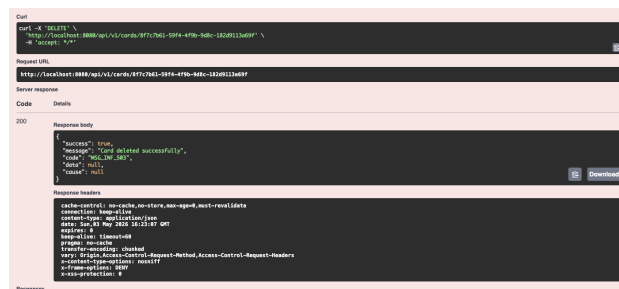
- * API: PATCH /api/v1/cards/:id
- * Input: listId (List đích), position (vị trí mới)
- * Pre-condition: Card và List đích tồn tại, user đã xác thực
- * Expected result: HTTP 200 OK, listId và position của card được cập nhật chính xác



Hình 5: Triển khai API Card TC20 - Drag and Drop

– TC21 - Xóa Card thành công

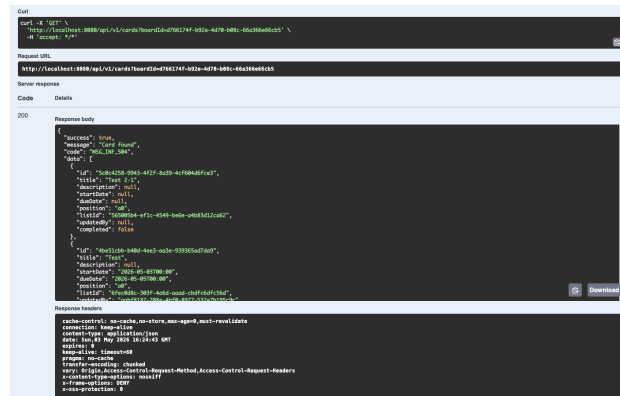
- * API: DELETE /api/v1/cards/:id
- * Pre-condition: Card tồn tại, user đã xác thực
- * Expected result: HTTP 200 OK



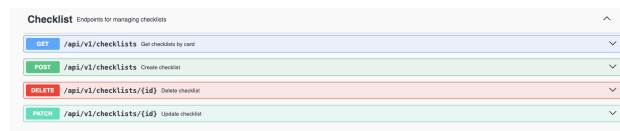
Hình 6: Triển khai API Card TC21

– TC22 - Lấy danh sách Card theo Board

- * API: GET /api/v1/cards?boardId=
- * Pre-condition: Board tồn tại, user đã xác thực
- * Expected result: HTTP 200 OK, response là danh sách card sắp xếp theo position (có thể là [] nếu chưa có dữ liệu)

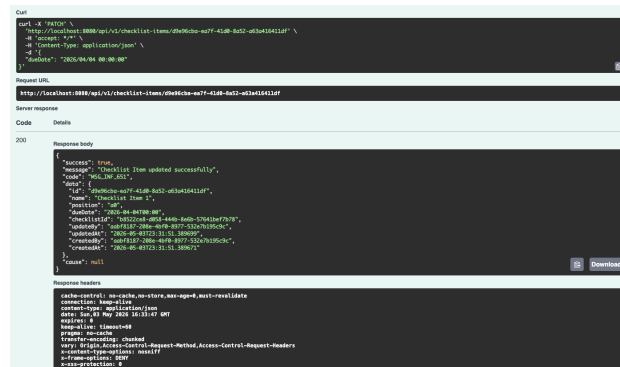


1.3 Quản lý Checklist



- Thiết kế entity/model và quan hệ cho Checklist
 - Xây dựng model Checklist: id, name, cardId, position, createdBy, updatedBy.
 - Xây dựng model ChecklistItem: id, name, checklistId, position, dueDate, isCompleted, createdBy, updatedBy.
 - Thiết kế quan hệ: Checklist thuộc về Card; ChecklistItem thuộc về Checklist.
- Xây dựng API tạo Checklist trong Card (POST /api/v1/checklists).
 - Nhận thông tin từ client: cardId, name, position.
 - Xác thực người dùng và kiểm tra Card tồn tại.
 - Trả về thông tin checklist vừa tạo.
- Xây dựng API cập nhật Checklist (PATCH /api/v1/checklists/:id).
 - Nhận thông tin cần cập nhật: name, position.

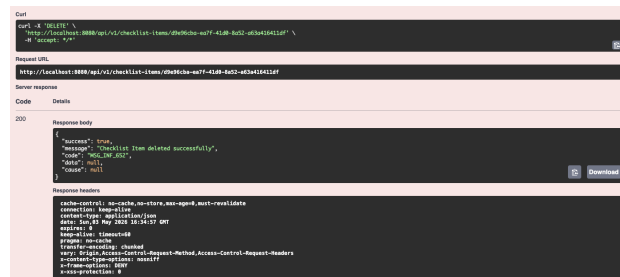
- Kiểm tra checklist tồn tại, trả về HTTP 404 nếu không tìm thấy.
- Trả về HTTP 200 và thông tin checklist sau khi cập nhật.
- Xây dựng API xóa Checklist (DELETE /api/v1/checklists/:id).
 - Kiểm tra checklist tồn tại, trả về HTTP 404 nếu không tìm thấy.
 - Trả về HTTP 200 khi xóa thành công.
- Xây dựng API lấy danh sách Checklist theo Card (GET /api/v1/checklists?cardId=).
 - Nhận cardId từ query parameter.
 - Trả về danh sách checklist thuộc Card kèm các item.
- Xây dựng API tạo Checklist Item (POST /api/v1/checklist-items).
 - Nhận thông tin từ client: checklistId, name, position, dueDate (tùy chọn).
 - Kiểm tra Checklist tồn tại.
 - Tạo item với trạng thái mặc định isCompleted = false.
 - Trả về thông tin item vừa tạo.
- Xây dựng API cập nhật Checklist Item (PATCH /api/v1/checklist-items/:id).
 - Nhận thông tin cần cập nhật: name, position, dueDate, isCompleted.
 - Kiểm tra item tồn tại, trả về HTTP 404 nếu không tìm thấy.
 - Trả về HTTP 200 và thông tin item sau khi cập nhật.
- Xây dựng API xóa Checklist Item (DELETE /api/v1/checklist-items/:id).
 - Kiểm tra item tồn tại, trả về HTTP 404 nếu không tìm thấy.
 - Trả về HTTP 200 khi xóa thành công.
- **Kết quả kiểm thử chức năng Checklist**
 - **TC23 - Tạo Checklist thành công**
 - * API: POST /api/v1/checklists
 - * Input: cardId, name = “Checklist 2”, position
 - * Pre-condition: Card tồn tại, user đã xác thực
 - * Expected result: HTTP 200 OK, response chứa checklistId, name, cardId, position



Hình 11: Triển khai API Checklist TC25

– TC26 - Xóa Checklist Item thành công

- * API: DELETE /api/v1/checklist-items/:id
- * Pre-condition: Item tồn tại, user đã xác thực
- * Expected result: HTTP 200 OK



Hình 12: Triển khai API Checklist TC26

– TC27 - Xóa Checklist thành công

- * API: DELETE /api/v1/checklists/:id
- * Pre-condition: Checklist tồn tại, user đã xác thực
- * Expected result: HTTP 200 OK



Hình 13: Triển khai API Checklist TC27

1.4 Kết luận

Trong tuần 5, nhóm đã hoàn thành toàn bộ các chức năng theo kế hoạch: triển khai đầy đủ API CRUD cho Card (TC17–TC22) bao gồm cả logic xử lý position phục vụ drag-and-drop, đồng thời hoàn thành API Checklist và Checklist Item (TC23–TC27). Hệ thống backend hiện đã đáp ứng đầy đủ các chức năng cốt lõi, sẵn sàng cho việc tích hợp với frontend trong các giai đoạn tiếp theo.

2 CHƯƠNG 2: KẾ HOẠCH PHÁT TRIỂN TUẦN 6

2.1 Mục tiêu

- Triển khai 3 màn hình xác thực người dùng (Sign Up, Confirm Sign Up, Sign In) và tích hợp với Authentication API.
- Triển khai màn hình Dashboard (danh sách Workspace và Board) và tích hợp với Workspace/Board API.
- Đảm bảo luồng điều hướng (navigation flow) giữa các màn hình hoạt động chính xác.
- Xử lý trạng thái loading, error và hiển thị thông báo phản hồi từ API cho người dùng.

2.2 Triển khai chi tiết

- **Triển khai màn hình Authentication**
Các bước thực hiện:

- Xây dựng màn hình **Sign Up** (/sign-up).
 - * Thiết kế form nhập thông tin: `fullName`, `email`, `password`.
 - * Tích hợp API `POST /api/v1/auth/sign-up`.
 - * Xử lý validation phía client trước khi gọi API.
 - * Điều hướng sang màn hình Confirm Sign Up sau khi đăng ký thành công.
 - * Hiển thị thông báo lỗi khi email đã tồn tại hoặc dữ liệu không hợp lệ.
- Xây dựng màn hình **Confirm Sign Up** (/confirm-sign-up).
 - * Thiết kế form nhập mã xác nhận được gửi qua email.
 - * Tích hợp API `POST /api/v1/auth/confirm-sign-up`.
 - * Hỗ trợ chức năng gửi lại mã qua API `POST /api/v1/auth/resend-code`.
 - * Điều hướng sang màn hình Sign In sau khi xác nhận thành công.
- Xây dựng màn hình **Sign In** (/sign-in).
 - * Thiết kế form nhập `email` và `password`.
 - * Tích hợp API `POST /api/v1/auth/sign-in`.
 - * Lưu trữ access token và refresh token (HTTP-only cookie) sau khi đăng nhập thành công.
 - * Điều hướng sang màn hình Dashboard sau khi đăng nhập thành công.
 - * Hiển thị thông báo lỗi khi sai thông tin đăng nhập.

Kết quả mong muốn:

- Ba màn hình Authentication hoạt động đầy đủ và tích hợp thành công với API.
- Luồng Sign Up → Confirm → Sign In hoạt động liên tục, không lỗi.
- Validation và thông báo lỗi hiển thị rõ ràng, đúng ngữ cảnh.

• Triển khai màn hình Dashboard

Các bước thực hiện:

- Xây dựng layout Dashboard hiển thị danh sách Workspace và Board của người dùng.
 - * Tích hợp API `GET /api/v1/workspaces` để lấy danh sách Workspace.

- * Tích hợp API `GET /api/v1/boards/:id` để lấy thông tin Board theo Workspace.
- * Hiển thị trạng thái rỗng (empty state) khi người dùng chưa có Workspace hoặc Board.
- Xây dựng chức năng tạo Workspace mới trực tiếp từ Dashboard.
 - * Tích hợp API `POST /api/v1/workspaces`.
 - * Lấy danh sách category từ API `GET /api/v1/workspace-categories` để hiển thị trong form.
 - * Cập nhật danh sách Workspace trên UI sau khi tạo thành công.
- Xây dựng chức năng tạo Board mới trong Workspace.
 - * Tích hợp API `POST /api/v1/boards`.
 - * Lấy danh sách ảnh nền từ API `GET /api/v1/images` để người dùng chọn `backgroundUrl`.
 - * Cập nhật danh sách Board trên UI sau khi tạo thành công.
- Xử lý xác thực phiên làm việc trên Dashboard.
 - * Gọi API `GET /api/v1/users/me` để xác minh phiên đăng nhập khi vào Dashboard.
 - * Tự động làm mới access token qua API `POST /api/v1/auth/refresh-token` khi token hết hạn.
 - * Điều hướng về màn hình Sign In nếu phiên làm việc không hợp lệ.

Kết quả mong muốn:

- Màn hình Dashboard hiển thị đầy đủ danh sách Workspace và Board, tích hợp thành công với API.
- Người dùng có thể tạo Workspace và Board mới trực tiếp từ Dashboard.
- Cơ chế xác thực phiên làm việc và tự động làm mới token hoạt động ổn định.
- Giao diện phản hồi trạng thái loading và lỗi rõ ràng trong mọi tình huống.